THE LAWS OF SOFTWARE TEAMS

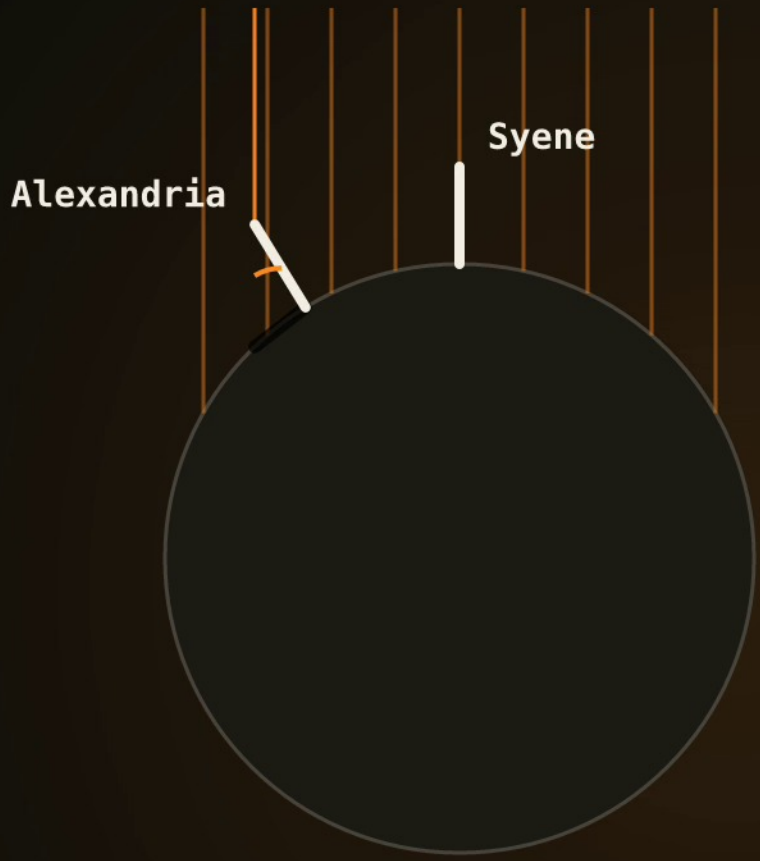
Gilb's law

Anything you need to quantify can be measured better than not at all

swipe →

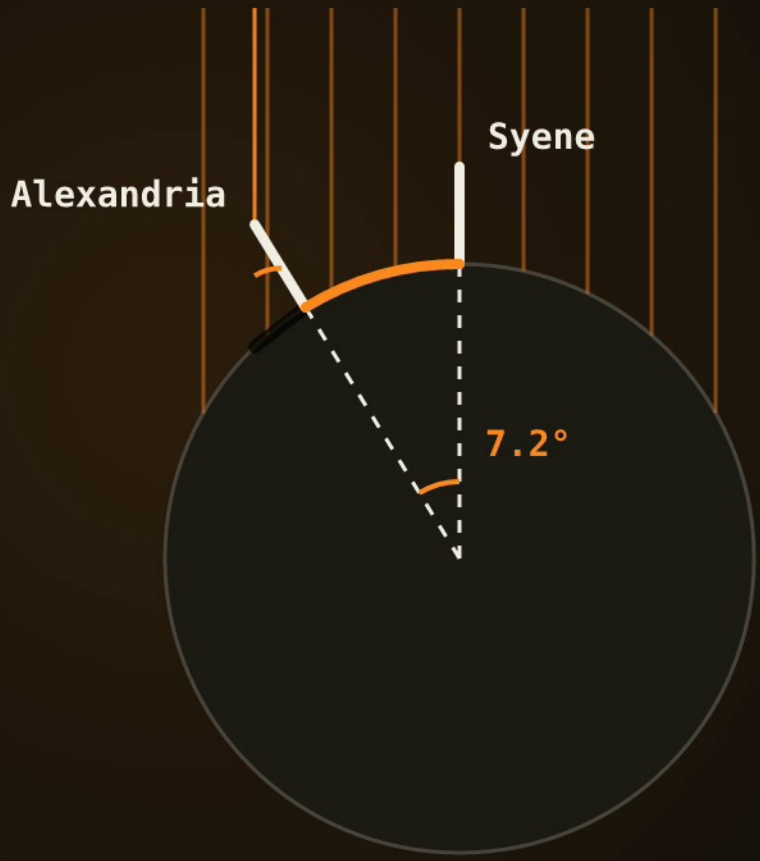
Ivan Rykov

He measured the Earth with a stick



~2,200 years ago. No satellites

Noon. No shadow in Syene 7.2° in Alexandria



7.2° = 1/50 of a circle

5,000 stadia × 50 = 250,000

"Anything you need to quantify can be measured in some way that is superior to not measuring it at all"

Gilb's law

Tom Gilb · named in Peopleware, DeMarco & Lister



Measuring ≠ certainty
It's less uncertainty

How long until a new hire ships?

weeks · example



first guess: 1-20 weeks

after 5 answers: 4-7 weeks

Illustrative example · idea from Douglas W. Hubbard

You need less data than you think



Pick 5 at random

94% chance the median is inside their range

Hubbard's Rule of Five · $1 - 2 \times (½)^5 = 93.75\%$

Two laws, one balance

GOODHART

Don't make it a target

GILB

Don't stop measuring

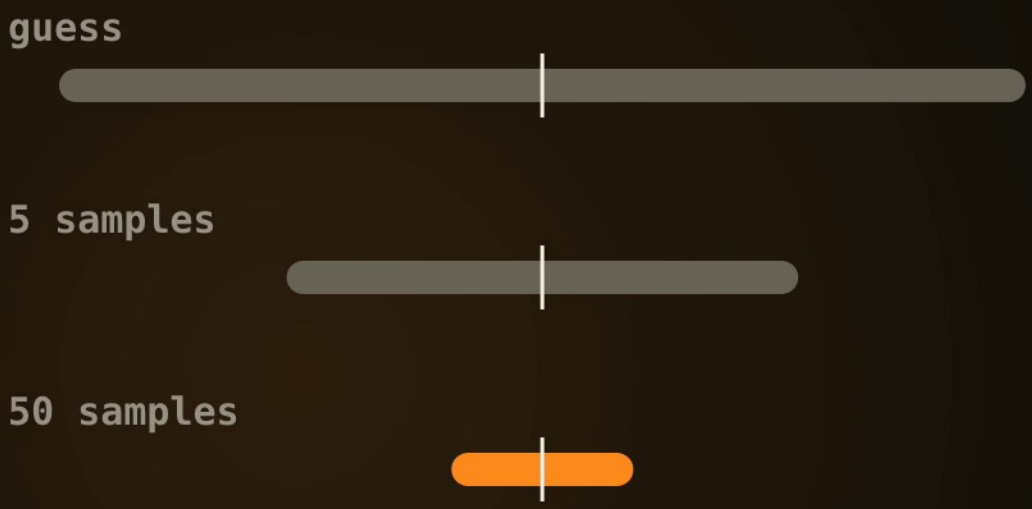
No data is worse

Start small

1. **Name the decision**
   the number should change
2. **Guess a range**
   e.g. 2-12 weeks
3. **Sample a little**
   and watch the range shrink

Hubbard: if it can't change a decision,
it isn't worth measuring

A rough number beats a confident guess



WHAT I TAKE FROM THIS

This is how I measure a team's work:

1. **Tasks, pull requests and activity are signals. I do not turn them into a score for one person, because a person does a lot of work that these numbers do not show. Strong people show in results and in how they act, not in a count**
2. **I always watch a small set of numbers. For delivery, I look at releases, failed releases and rollbacks, and errors in Sentry. For systems, I look at SLOs, the speed of key APIs and the cost per request or per client. For the flow of work, I look at review time, tasks in progress and backlog growth. I also track incidents and recovery time. I check lead time from time to time, and unplanned work when the team starts to slow down**

WHAT I TAKE FROM THIS

3. **Hard things get rough numbers. Tech debt can be a ratio: how many new debt items appear for each finished task. Team mood comes mostly from what a lead sees: how often people take the initiative, suggest solutions and speak in calls. A mood meter at retros helps too. With no data, I start rough and collect it week by week**

WHAT I TAKE FROM THIS

4. I used to write my 1:1 notes by hand. About two years ago, the team itself suggested recording the calls and making transcripts automatically. Some calls are not recorded, and for them I write notes by hand. The recordings, transcripts and notes are only for me: I use them to improve the team and the product. Later I can group what different people said and see which problems come up again. A survey works when it is rare, has a clear goal and asks specific questions. Once I made an interactive survey, with a bit of humour, about our platform and how easy our tools were to use. It asked what to improve, change or add, and what to keep. Everyone completed it, and it gave us a clear list of changes

WHAT I TAKE FROM THIS

5. A metric is useful when a decision follows from it. When we had only two releases a month, we measured how long a release took and the dependencies between services, and then fixed what slowed the releases down. A cost breakdown by service showed which services to optimise first. Usage data showed which old API versions we could switch off. Clients got one or two months of notice first. Zero usage in PostHog and logs is an argument for removing a feature. We once found several such features: some were removed, and some stayed. The same kind of data also showed that a cold-start problem we planned to solve did not exist
6. A target needs a requirement behind it. Once we measured about 20 requests per second and decided that it should be 50. Nobody asked for it, and we spent time tuning. An SLO without a requirement has the same problem

WHAT I TAKE FROM THIS

7. **If nobody opens a dashboard, we delete it or explain why it exists. A good one spreads by itself: I made a dashboard of pipeline times for my on-call shifts, and everyone started to use it**

With AI tools, we measured the number and size of pull requests, review time, time to merge, fixes after these pull requests and rollbacks. The number and size of pull requests grew a lot, and review became the bottleneck. So we added automatic review and more checks before a person reviews the code

Sources: DeMarco and Lister, Peopleware · Douglas W. Hubbard, How to Measure Anything · Eratosthenes

next: Goodhart's law